



Backend System Design

Terratrade API | Java 21, Spring Boot, PostgreSQL

Author: Bashir Muhammed

Document type: Backend architecture reference

API base path: `/api/v1`

Deployment: Docker on Fly.io | Database: PostgreSQL + PostGIS (Supabase)



1. Purpose and Scope

Terratrade is a cross-border agricultural trade platform. The backend exposes a REST API that supports three user roles: farmer, aggregator, and international buyer. It handles harvest listings, geo-based sourcing, export batching, compliance document review, RFQ matching, escrow payments, and activity notifications.

This document describes the backend only: architecture, modules, data model, security, integrations, and deployment. It is intended for engineers reviewing the system design or onboarding to the codebase.

2. System Context

The API sits between client applications and external services. All business state is stored in PostgreSQL. File uploads are served through a storage abstraction. Payment and document analysis are delegated to integration services.

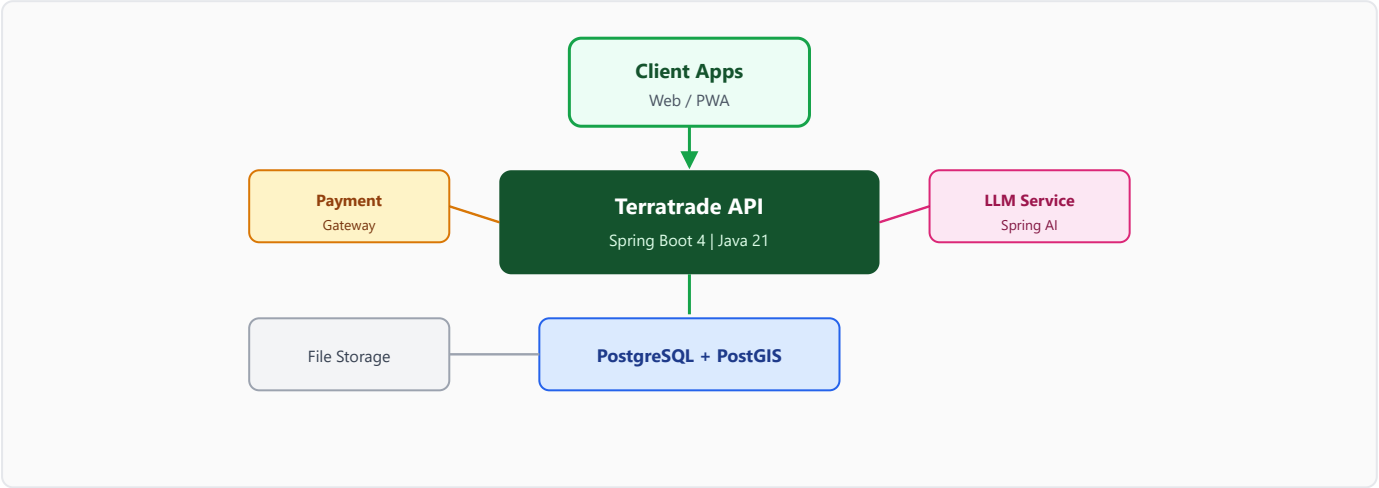


Figure 1. System context: clients call the API; the API persists data and calls external services.

3. Technology Stack

Layer	Technology	Role
Runtime	Java 21	Language and JVM runtime
Framework	Spring Boot 4.0.6	REST API, DI, configuration
Persistence	Spring Data JPA, Hibernate	ORM and repositories
Database	PostgreSQL + PostGIS	Relational data and geo queries
Migrations	Flyway	Versioned schema (V1, V2)
Security	Spring Security, JWT (JJWT)	Stateless authentication
API docs	Springdoc OpenAPI	Swagger UI at /swagger-ui
Integrations	RestClient, Spring AI	Payments, document analysis
Build	Maven	Dependency management and packaging
Deploy	Docker, Fly.io	Container hosting with volume for uploads

4. Layered Architecture

The backend follows a conventional Spring layered design. Controllers handle HTTP. Services hold business rules and orchestration. Repositories access the database. Integration packages isolate third-party calls. DTO records decouple API contracts from JPA entities.

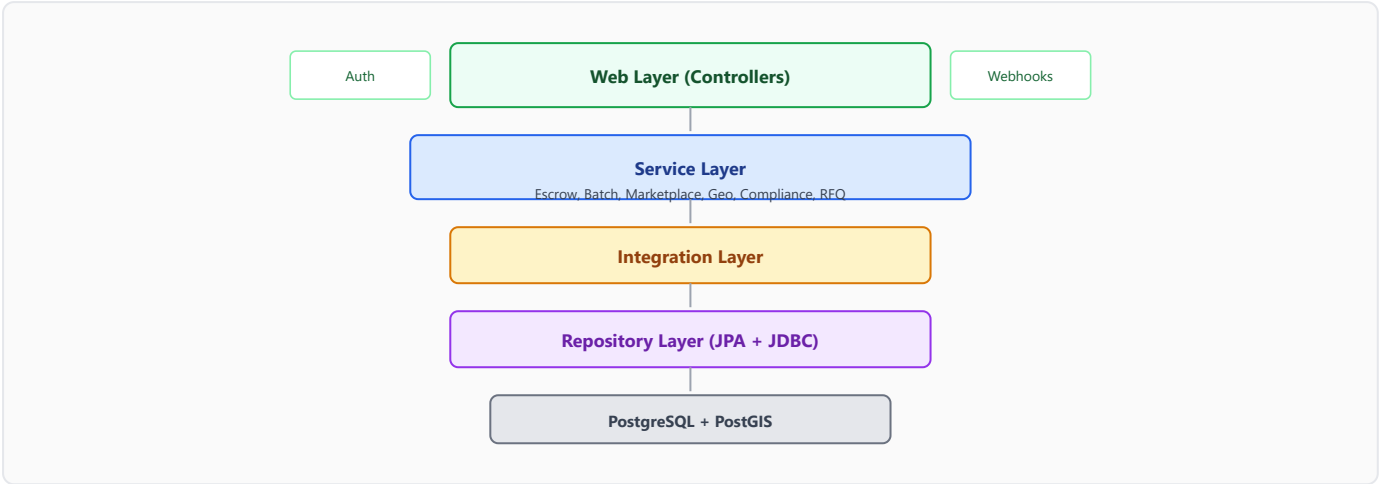


Figure 2. Layered backend architecture.

5. Package Structure

Package	Responsibility
web	REST controllers, request mapping, validation entry points
service	Business logic, transactions, orchestration across repositories
repository	Spring Data JPA interfaces and custom queries
domain.entity	JPA entity classes mapped to tables
domain.enums	Status enums: harvest, batch, escrow, compliance
dto	Java records for API request and response shapes
security	JWT filter, security config, user principal
integration.opay	Checkout and payout HTTP clients
integration.gemini	Compliance document audit via Spring AI
exception	Global exception handler, ApiException
seed	DemoDataSeeder for pitch and local testing

6. API Surface (by domain)

Prefix	Controller	Domain
/api/v1/auth	AuthController	Register, login, JWT issuance
/api/v1/farmer	FarmerController	Plots, harvests, watches, sales
/api/v1/aggregator	AggregatorController	Sourcing, cart, batches, RFQs
/api/v1/buyer	BuyerPortalController	Marketplace, orders, watches
/api/v1/rfqqs	BuyerController	RFQ CRUD, interest, award
/api/v1/escrow	EscrowController	Local and global escrow flows
/api/v1/batches	BatchController	Batch build, transparency, scorecard
/api/v1/batches/{id}/documents	ComplianceController	Document upload and audit
/api/v1/demo	DemoController	Demo data reset and seed (public)
/api/v1/webhooks	OpayWebhookController	Payment callback handling
/api/v1/catalog	CatalogController	Crops, states (public)
/api/v1/files	FileController	Serve uploaded documents and images

7. Security Architecture

Authentication is stateless. The client sends a Bearer JWT on protected routes. Spring Security validates the token, loads the user, and attaches a role-aware principal. Passwords are hashed with BCrypt at registration.

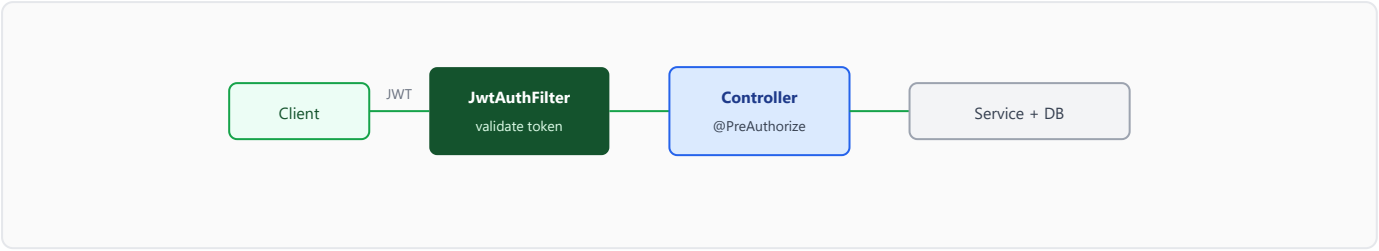


Figure 3. Request path for authenticated API calls.

Public endpoints (no JWT)

- /api/v1/auth/** , /api/v1/demo/** , /api/v1/catalog/**
- /api/v1/webhooks/** (payment callbacks)
- /actuator/health , Swagger UI, GET /api/v1/files/**

Role checks are enforced in services and through method security where needed. CORS origins are configured via environment variable CORS_ORIGINS .

8. Data Model Overview

The schema is defined in Flyway migrations. V1 creates the baseline tables. V2 adds PostGIS boundaries, plot location triggers, and geo alert columns. Hibernate validates the schema at startup (ddl-auto=validate).

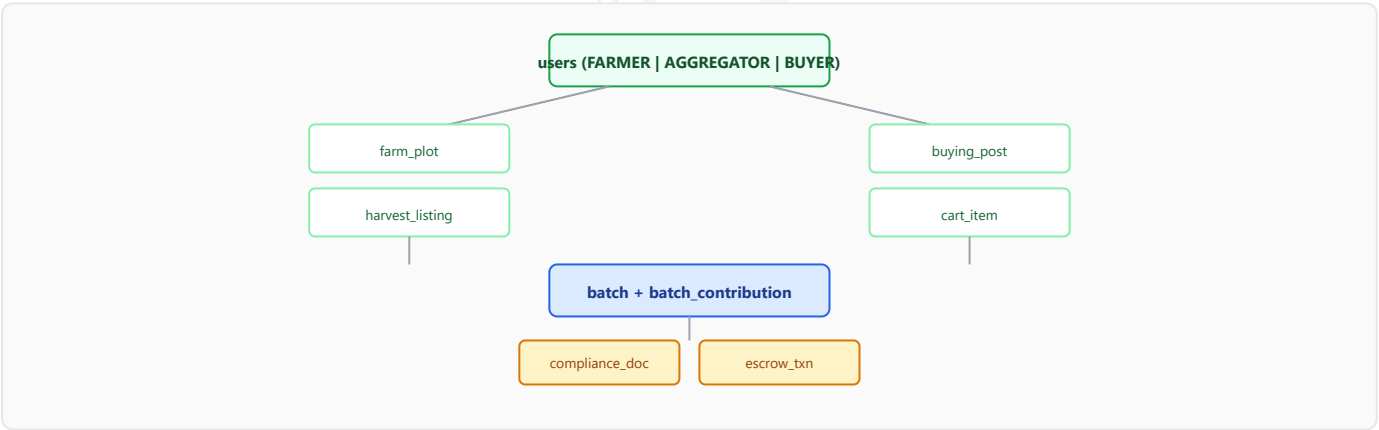


Figure 4. Core entity relationships (simplified).

Table group	Tables
Catalog	crop_catalog, state_region
Users	users
Farmer	farm_plot, harvest_listing, harvest_attachment, demand_watch
Aggregator	buying_post, cart_item, batch, batch_contribution
Buyer	buyer_rfq, rfq_interest, buyer_batch_watch, sample_request
Payments	escrow_transaction, escrow_cart_link, payment_payout
Compliance	compliance_document, rfq_document
Audit	platform_activity, audit_log, harvest_change_log

9. Core Workflow: Local Escrow (Aggregator to Farmer)

Local escrow protects farmers and aggregators during domestic sourcing. The aggregator adds harvests to a cart, funds escrow through the payment gateway, and confirms delivery. The service then triggers split payouts to farmers.

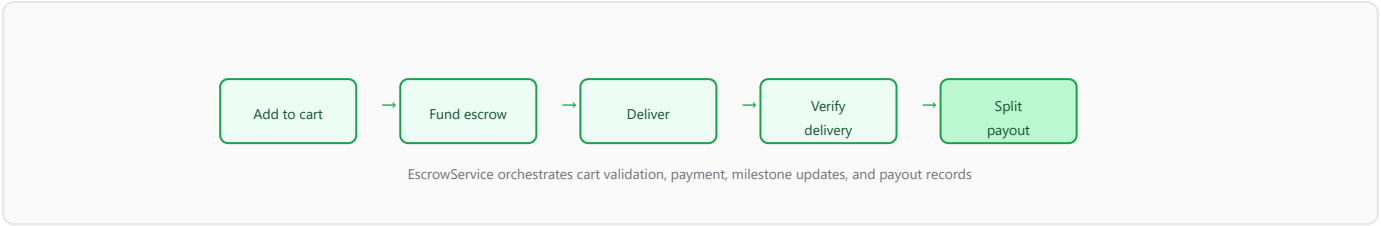


Figure 5. Local escrow milestone flow.

Escrow milestones progress through BOOKED, DELIVERED, COMPLIANT, and RELEASED. Each transition is logged in `audit_log`. Mock mode allows demo checkout without live payment credentials.

10. Core Workflow: Export Batch and Compliance

After harvests are purchased, the aggregator builds a batch from paid cart items. Documents are uploaded per crop checklist. The compliance service sends PDFs to the LLM integration, stores extracted fields and flags, and computes a scorecard. Batches reach EXPORT_READY when the score threshold is met.

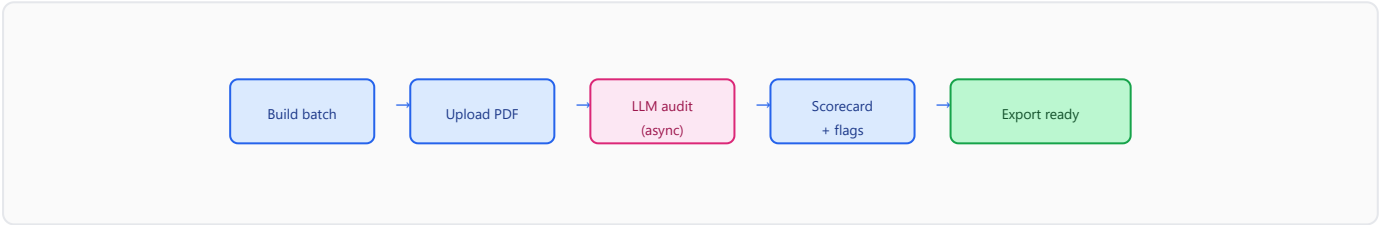


Figure 6. Compliance vault processing pipeline.

11. Geo and Traceability

`GeoService` uses JDBC against PostGIS for operations that go beyond standard JPA queries:

- Plot pin validation against state boundary polygons
- Radius harvest search with `ST_DWithin` from aggregator hub
- Batch transparency footprint (convex hull across contributing plots)
- Geo-filtered demand and batch watches

A database trigger keeps `farm_plot.location` geometry in sync with latitude and longitude. This supports origin proof for export compliance and buyer transparency maps.

12. Integration Layer

Payment gateway (OpayCheckoutService, OpayPayoutService)

- Creates escrow checkout sessions
- Processes split payouts to farmer wallets
- Mock mode for demo and local dev
- Webhook controller receives payment status callbacks

Document analysis (ComplianceAuditService)

- Spring AI ChatClient sends PDF + prompt
- Extracts certificate fields as JSON
- Scores against batch and RFQ criteria
- Runs asynchronously with @Async

File storage (StorageService)

Abstraction over local disk (Fly volume at /app/uploads) with optional S3-compatible endpoint. Used for harvest photos, compliance PDFs, and RFQ documents. Served via FileController.

13. Key Service Responsibilities

Service	Responsibility
AuthService	Registration, login, JWT creation, password encoding
MarketplaceService	Harvest listings, plots, buying posts, cart operations
EscrowService	Local and global escrow lifecycle, payouts
BatchService	Batch creation, contributions, transparency response
ComplianceService	Document upload, audit trigger, scorecard assembly
RfqBoardService	RFQ board, supplier interest, award, PO generation
RfqMatchService	Batch fit scoring against buyer RFQ criteria
GeoService	PostGIS queries for boundaries, radius, footprint
ActivityService	Platform notifications and unread counts
AuditLogService	Immutable audit trail for escrow and compliance events

14. Deployment Architecture

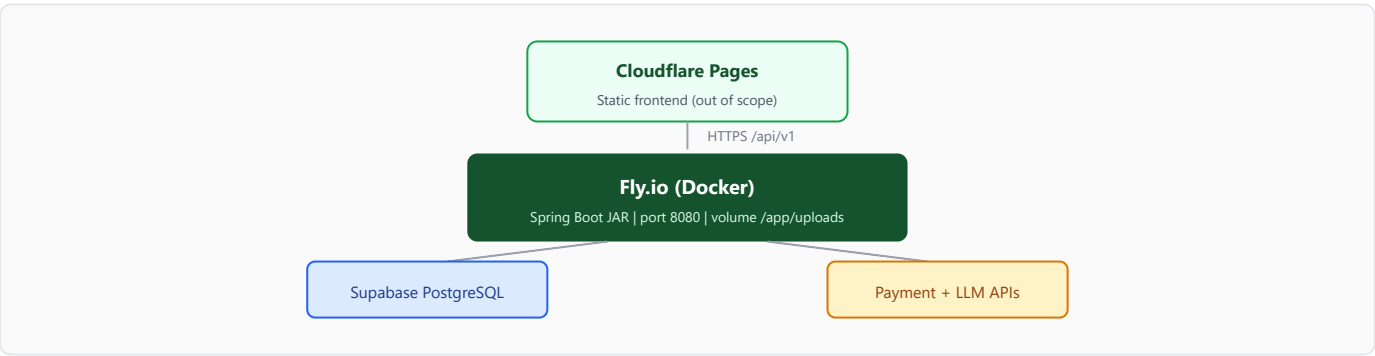


Figure 7. Production deployment topology (backend focus).

Configuration is externalised through environment variables: database URL, JWT secret, CORS origins, payment keys, and storage paths. Health checks expose `/actuator/health` for platform monitoring.

15. Design Decisions

Decision	Rationale
Stateless JWT auth	Simple scaling on Fly.io without session store
Flyway over ddl-auto update	Reproducible schema across local, demo, and production
DTO records separate from entities	Stable API contracts, no lazy-loading leaks
Integration package isolation	Payment and LLM clients swappable without touching domain logic
JDBC for geo queries	PostGIS functions are clearer and more efficient in native SQL
Mock payment mode	Enables full demo flow without live merchant credentials
Per-crop compliance catalog	Sesame, cocoa, and ginger each carry distinct document rules
Audit log table	Traceable record for escrow and compliance state changes

16. Summary

Terratrade backend is a role-based Spring Boot API that models the full agricultural export chain in PostgreSQL, secures access with JWT, integrates payment and document analysis services, and uses PostGIS for location-aware sourcing and traceability. The design prioritises clear layering, versioned schema, and demo-ready flows that map directly to real farm-to-export business steps.

Repository path	backend/src/main/java/com/terratrade/
Migrations	backend/src/main/resources/db/migration/
API docs	/swagger-ui (when backend is running)
Health	/actuator/health